МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра "ПОВТ и АС"

Платформа JavaFX

Методические указания к лабораторной работе по дисциплинам "Объектно-ориентированное программирование", "Программирование под платформу Java"

Ростов-на-Дону 20 г.

Составитель: к.ф.-м.н., доц. Габрельян Б.В.
УДК 512.3

Платформа JavaFX: методические указания – Ростов н/Д: Издательский центр ДГТУ, 20 . – 8 с.

Рассмотрена программная среда JavaFX: создание элементов графического пользовательского интерфейса, использование 2D и 3D графики, мультимедиа. Даны задания по выполнению лабораторной работы. Методические указания предназначены для студентов направлений 090304 "Программная инженерия", 020303 "Математическое обеспечение и администрирование информационных систем".

Ответственный редактор:

Издательский центр ДГТУ, 20

Библиотека JavaFX обеспечивает поддержку элементов графического пользовательского интерфейса (GUI), 2D- и 3D-графики, отображения мультимедиа контента, анимации. Позволяет использовать компоненты библиотеки Swing. Дает возможность отделить проектирование внешнего вида приложения от его бизнес-логики за счет использования специального языка разметки FXML. При этом описание элементов GUI, их размеров, положения и других характеристик размещается в текстовом fxml-файле, а логика работы программы, в том числе обработчики событий, связанных с элементами GUI размещается в java-файлах. Формирование fxml-файлов можно поручить специальной программе - какому-нибудь визуальному редактору и тогда внешний вид программы можно создавать вообще не программируя. В JavaFX есть такой редактор - это SceneBuilder. Кроме того, оформление внешнего вида компонентов можно задавать в программе так, как это делается в каскадных таблицах стилей CSS или же просто указывать нужную таблицу. Например,

```
Label label = new Label();
        label.setStyle("-fx-text-fill: #ff0000; -fx-font-size: 12pt;" +
        "-fx-font-family: 'Times New Roman';");
```

или

```
        Label label = new Label();
        label.getStylsheets().add(
                getClass().
                        getResource("mystyle.css").toExternalForm()
        );
        label.getStyleClass().add("mylabel");
```

при наличии файла mystyle.css со следующим содержимым

```
.mylabel {
 -fx-text-fill: #ff0000;
```

```
 -fx-font-size: 12pt;
 -fx-font-family: 'Times New Roman';
}
```

Задает для метки label цвет (красный), размер шрифта (12) и семейство шрифтов (Times New Roman).

1. Подключение JavaFX

Скачать с сайта https://gluonhq.com/products/javafx/ JavaFX SDK, JavaFX jmods и JavaFX Documentation. Современные версии JavaFX построены по модульному принципу, поэтому как при компиляции, так и при запуске на выполнении нужно указывать информацию, позволяющую компилятору и интерпретатору JVM найти нужные модули. Инструкции по подключению библиотеки JavaFX в VS Code, IntelliJ Idea, NetBeans и Eclipse см. в файле lab1appendices.pdf.

2. Общая схема JavaFX приложения

В каждом приложении с графическим пользовательским интерфейсом явно или неявно запускается бесконечный цикл, в котором ожидаются, получаются и обрабатываются различные события. Это могут быть события, связанные с внешними устройствами - мышью, клавиатурой, джойстиком и т.п., события от таймеров или программные события, генерируемые в самом приложении. В современных библиотеках, например, JavaFX, события обрабатываются в собственном потоке команд, отличном от других потоков, например, от потока, в котором выполняется метод main. В JavaFX поток обработки событий называется JavaFX Application Thread. Так же, как и в Swing, в JavaFX предполагается, что изменения элементов GUI размещенных в окне программы, должны проводиться только в потоке Application Thread. Для

этого метод, выполняющий изменения "оборачивается" в объект, представляющий событие и помещается в очередь обработки событий. Очередь событий обрабатывается в потоке Application Thread. Вызвать метод в этом потоке можно используя статический метод void runLater(Runnable r) класса Platform. Функциональный интерфейс Runnable содержит описание метода void run(). Таким образом, вызвать из потока, отличного от Application Thread свой метод так, чтобы он выполнялся в потоке Application Thread можно, например, так

```
Platform.runLater(
        new Runnable() {
                @Override
                public void run() {
// это сам вызываемый метод или же он должен вызываться отсюда
                        …
                }
        }
);
```

или, используя лямбда-функцию

```
Platform.runLater( () -> Platform.exit() );
```

или, используя указатель на метод

```
Platform.runLater( Platform::exit );
```

Класс Platform содержит также метод exit(). Вызов Platform.exit(); - это стандартный способ завершения JavaFX приложения в нужной его точке, а не в результате нормального завершения метода main.

JavaFX-приложение должно быть представлено классом, производным от стандартного абстрактного класса Application из пакета javafx.application. JavaFX создает поток обработки событий, очередь событий, а класс Application дает возможность запустить бесконечный цикл обработки событий. Для этого

в классе Application реализован статический метод void launch(). Жизненный цикл приложения в классе Application задают методы void init(), void stop() и абстрактный метод void start(Stage stage). Метод init выполняется в том же потоке, в котором вызван метод launch, а методы start и stop в потоке Application Thread. Таким образом, каждое JavaFX приложение должно быть представлено классом-наследником Application, в котором определен как минимум метод start. Кроме того, в методе main (точке входа в программу) нужно вызвать статический метод launch класса Application.

```
package mypackage; import javax.application.Application;
import javafx.stage.Stage;
…
public class MyJavaFXApp extends Application {
        …
        @Override
        public void start(Stage stage) {
                …
        }
        …
        public static void main(String[] args) {
                Application.launch(); /* или просто launch(); так как класс
MyJavaFXApp - наследник Application */
        }
}
```

Класс Stage представляет в программе окно верхнего уровня (подобно, например, JFrame в Swing). Объект, представляющий главное окно программы создает JavaFX и передает его методу start(). Если понадобятся другие подобные окна их нужно будет создавать явно. Используя вслед за создателями

JavaFX театральную терминологию, мы можем считать, что наша программа с помощью элементов графического пользовательского интерфейса устраивает некоторое шоу для пользователей программы, подобно тому как в театре на какой-то из его сцен актеры с помощью реквизита устраивают представление для зрителей. Тогда общую логику организации программы можно описать так: JavaFX дает нам театр (объект класса Stage) в котором мы должны указать (задать) сцену (объект класса Scene), на этой сцене мы должны разместить актеров и реквизит (элементы GUI) которые вместе и организуют представление для пользователей программы. Элементы GUI, расположенные на сцене логически связаны друг с другом. Должен быть корневой элемент (контейнер), который добавляется на сцену при ее создании, остальные элементы GUI добавляются в корневой элемент. Среди этих элементов могут быть контейнеры, содержащие другие элементы. Тем самым элементы на сцене образуют некоторое дерево (граф сцены). При создании сцены задается также ее размер.

```
package mypackage; import javax.application.Application; import
javafx.stage.Stage;
import javafx.scene.Scene; import javafx.scene.layout.VBox; import
javafx.scene.control.Label;

public class MyJavaFXApp extends Application {
        @Override
        public void start(Stage stage) {
/* корневой элемент - контейнер, поддерживающий вертикальное располож-
ние добавленных в него компонентов, 5 - расстояние (в пикселях) по вертика-
ли между компонентами */
                VBox root = new VBox(5);
```

/* для сцены задаются корневой элемент, вертикальный и горизонтальный размеры в пикселях */

```
            Scene scene = new Scene(root, 320, 200);
```

/* элементы GUI, размещаемые в корневом элементе, метки - аналог JLabel в Swing */

```
            Label hello = new Label("Hello");
            Label world = new Label("World");
```

/* добавление компонентов в корневой элемент */

```
            root.getChildren().add(hello);
            root.getChildren().add(world);
            // или root.getChildren().addAll(hello, world);
```

/* задание заголовка главного окна, сцены для него и отображение окна на экране */

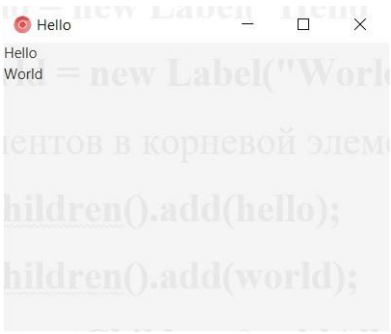
```
            stage.setTitle("Hello");
            stage.setScene(scene);
            stage.show();
        }
 }
```

Для Stage можно задавать различные стили, определяемые значениями перечислимого типа javafx.stage.StageStyle: DECORATED, UNIFIED, UNDECORATED, TRANSPARENT, UTILITY. Стили определяют будут ли у окна декораторы - заголовок, кнопки максимизации, минимизации, закрытия (см. документацию [5]). Если декораторы поддерживаются, можно задать пиктограмму для заголовка окна - изображение размером 32x32 пикселя в графическом формате (jpg, png, gif, …). Например,

```
stage.getIcons().add(
            new Image(
```

```
                getClass().getResource ("icon.png").toExternalForm();
            )
        );
```

Кроме того, можно задать степень прозрачности окна, от 0 (полностью прозрачное) до 1 (полностью непрозрачное). Например, stage.setOpacity(.95);



Здесь задана иконка красного цвета для заголовка окна и на 5% прозрачное окно - сквозь окно виден текст, поверх которого оно расположено.

С каждым контейнером компонентов связан собственный менеджер компоновки. *javafx.scene.Group* и *javafx.scene.layout.Pane* не выполняют автоматического размещения компонентов (абсолютный менеджер компоновки в терминологии Swing). Поэтому нужно размещать все компоненты явно.

```
Label hello = new Label("Hello");
hello.setLayoutX(20);
hello.setLayoutY(50);
```

Размещаем метку так, чтобы левый верхний угол компонента оказался в точке с координатами (20, 50). Размеры компонента можно задать с помощью методов: void setPrefWidth(double w), void setPrefHeight(double h) или void setPrefSize(double width, double height) - предпочтительные размеры, void setMinWidth(double w), void setMinHeight(double h) или void setMinSize(double width, double height) - минимальные размеры, void setMaxWidth(double w), void setMaxHeight(double h) или void setMaxSize(double width, double height) - максимальные размеры.

javafx.scene.layout.VBox размещает компоненты последовательно – вертикально, вновь добавленный компонент располагается под теми, которые были добавлены прежде.

javafx.scene.layout.HBox размещает компоненты последовательно – горизонтально, вновь добавленный компонент располагается справа от тех, которые были добавлены прежде. *javafx.scene.layout.FlowPane* (аналог FlowLayout из Swing) размещает компоненты последовательно - горизонтально, если задана горизонтальная ориентация (значение HORIZONTAL перечисления Orientation, это значение по умолчанию), или вертикально (Orientation.VERTICAL). Если элементы не помещаются в текущей строке, они переносятся на следующую.

javafx.scene.layout.BorderPane (аналог BorderLayout из Swing) размещает компоненты в одной из пяти областей: верхней, нижней, левой, правой или в центре. Для этого в классе BorderPane предусмотрены специальные методы void setTop(компонент), void setBottom(комопнент), void setLeft(комопнент), void setRight(комопнент), void setCenter(компонент).

javafx.scene.layout.GridPane (аналог GridBagLayout из Swing) размещает компоненты в сетке. Чтобы добавить компонент в ячейку сетки на пересечении строки row и столбца col нужно вызвать статические методы класса GridPane setRowIndex и setColumnIndex,

```
GridPane.setRowIndex(компонент, row);
GridPane.setColumnIndex(компонент, col);
```

или использовать специальную форму метода add

```
GridPane gpane = new GridPane();
gpane.add(компонент, col, row);
```

Кроме того, можно использовать ограничения, позволяющие размещаемому в сетке компоненту занимать не одну ячейку, а несколько рядом расположенных ячеек. Можно также задавать различную ширину для разных столбцов сетки.

Pane является базовым классом для всех перечисленных выше контейнеров, кроме Group. В свою очередь Pane - класс-наследник класса Region. В Region определено свойство padding, задающее отступы содержимого, помещаемого в контейнер от границ контейнера. Задать отступы можно с помощью метода Region void setPadding(Insets i). Значения отступов содержатся в объекте Insets. Например,

```
FlowPane controlPane = new FlowPane();
double topMargin=10, leftMargin=15, bottomMargin=20, rightMargin=25;
controlPane.setPadding(
        new Insets(topMargin, rightMargin, bottomMargin, leftMargin)
);
```

Контейнеры с автоматическим размещение компонентов (из перечисленных все, кроме Group и Pane) позволяют задавать свободное пространство (spacing) (или зазор - gap) между компонентами. Для VBox и HBox его (соответственно по вертикали или по горизонтали) можно указать как аргумент конструктора:

```
VBox vbox = new VBox(5);
```

или с помощью метода void setSpacing(double val)

```
VBox vbox = new VBox();
vbox.setSpacing(5);
```

Для FlowPane, BorderPane и GridPane реализованы методы void setVgap(double gap) и void setHgap(double gap).

Для компонентов, размещаемых в контейнере можно задавать различное выравнивание: по базовой линии (если у компонента есть текстовая метка), по правому, левому краю, по центру, см. в документации [5] информацию по перечислимому типу Pos. Например,

```
HBox hbox = new HBox(3);
hbox.setAlignment(Pos.CENTER);
```

Другие варианты менеджеров компоновки см. в документации [5].

3. Элементы GUI

JavaFX поддерживает все основные элементы GUI [1]. Базовым классом для всех узлов графа сцены является абстрактный класс javafx.scene.Node. Этот класс задает поддержку обработки событий от мыши и клавиатуры. К объектам Node можно применять трансформации: поворот (rotation), перемещение (translation), масштабирование (scaling) и сдвиг/перекос (shearing). Можно также добавлять эффекты (тень, отражение, размытие и т.п.) с помощью метода setEffect. Можно задавать размеры и размещение компонентов в контейнере. Можно задавать оформление компонента в стиле CSS напрямую, или с указанием CSS файла. Для этого с каждым узлом можно связать идентификатор id и класс стиля styleClass, которые будут использоваться каскадной таблицей стилей CSS для идентификации узла, к которому должен быть применен соответствующий стиль.

Чаще всего таблица стилей задается для контейнера или для всей сцены

```
VBox box = new VBox(10.0);
box.getStylesheets().add(
        getClass().getResource("mystyle.css").toExternalForm());
Label label = new Label();
label.getStyleClass().add("mylabel");
```

Если в CSS-файле, заданном для контейнера или сцены использовать некоторый предопределенный селектор (класс), например, label, то соответствующие стили получат все компоненты этого типа в данном контейнере или на данной сцене. Например,

```
VBox root = new VBox(5);
Label hello = new Label("Hello");
```

```
Label world = new Label("World");
root.getChildren().addAll(hello, world);
Scene scene = new Scene(root, 600, 500);
scene.getStylesheets().add(
        getClass().getResource("mystyle.css").toExternalForm());
```

Все метки получат одинаковые стили.

mystyle.css

```
.label {
 -fx-text-fill: #ff0000;
 -fx-font-size: 12pt;
 -fx-font-family: 'Times New Roman';
}
```

Для узла можно задать некоторые пользовательские данные void setUserData(Object o), которые в дальнейшем можно получить с помощью метода Object getUserData(). Можно влиять на видимость узла - void setVisible(Boolean b).

Базовые элементы GUI подобны таковым из Swing: например, Label (JLabel), Text (JTextField) и т.д. [1]

Есть также как в Swing, компоненты для выбора цвета javafx.scene.control.ColorPicker, файла javafx.stage.FileChooser, даты javafx.scene.control.DatePicker, стандартных диалоговых окон с сообщениями javafx.scene.control.Alert.

```
import javafx.scene.control.*; // ColorPicker, Alert
import javafx.stage.FileChooser;
import javafx.scene.paint.Color;
import java.io. File;
import java.io.IOException;
…
FileChooser fc = new FileChooser();
c.getExtentionFilters().add( /* фильтр - какие фалы отображать (только с расширением png, jpg, или gif в данном примере) */
new FileChooser.ExtentionFilter("Image files", "*.png", "*.jpg", "*.gif")
);
```

```
File file = fc.showSaveDialog(null); // сохранить файл с заданным именем
…
```

чтобы открыть файл, например, текстовый и прочитать его содержимое

```
…
FileChooser fc = new FileChooser();
fc.getExtentionFilters().add(
        new FileChooser.ExtentionFilter("Text files", "*.txt")
);
File file = fc.showOpenDialog(null);
if(file != null) {
        java.util.Scanner scan = new Scanner(file);
/*
можно задать локаль, определяющую, в частности, формат даты, времени, записи действительных чисел (1.5 или 1,5), например, scan.useLocale(Locale.ENGLISH); если нужен формат 1.5, в русской локали 1,5
*/
        String str = null;
        if(scan.hasNext()) str = scan.next(); // до первого пробела
        if(scan.hasNextLine()) str = scan.nextLine(); // до '\n'
        int x = 0;
        if(scan.hasNextInt()) x = scan.nextInt();
        double d = 0.;
        if(scan.hasNextDouble()) d = scan.nextDouble();
/* в русской локали 1.5 не будет восприниматься как double, но 1,5 воспринимается как double */
        …
}
```

…

ColorPicker cp = new ColorPicker();

cp.setValue(Color.BLACK); /* начальное значение цвета, по умолчанию WHITE */

…

Color color = cp.getValue();

…

Alert info = new Alert(Alert.AlertType.INFORMATION); /* информационный диалог с заголовком окна (title), заголовком сообщения (header text) и самим сообщением (content text); содержит иконку с изображением восклицательного знака и кнопку "OK" */

info.setTitle("Информация");

info.setHeaderText("Как пользоваться программой");

info.setContentText(

"Пользоваться этой программой также легко и приятно,\n" +

"как и всякой другой"

);

Для создания и использования меню предусмотрены классы MenuItem, Menu, MenuBar (пакет javax.scene.control). Простейшую полосу меню (MenuBar) с несколькими выпадающими меню (Menu), содержащими несколько пунктов меню (MenuItem) можно создать следующим образом:

BorderPane root = new BorderPane(); // корневой элемент сцены

Menu menu1 = new Menu("_File"); /* символ, перед которым есть знак подчеркивания будет использоваться для мнемоники - сочетания клавиш для быстрого вызова меню */

menu1.setMnemonicParsing(true); // использовать мнемонику: Alt+F

MenuItem menu1Item1 = new MenuItem("Open…");

```
menu1Item1.setOnAction(
        e->{…}      // код для обработки события выбора пункта меню
);
MenuItem menu1Item2 = new MenuItem("Exit");
menu1Item1.setOnAction(
        e->{ Platform.exit(); }    // завершить работу программы
);
/* добавить пункты меню в меню, между ними сепаратор -
разделительная линия */
menu1.getItems().
addAll(menu1Item1, new SeparatorMenuItem(), menuItem2);
// так же для второго меню
Menu menu2 = new Menu("_Help");
…
// полоса меню
MenuBar mbar = new MenuBar();
mbar.getMenus().addAll(menu1, menu2);
// добавить MenuBar в верхнюю часть корневого элемента сцены
root.setTop(mbar);
```

В JavaFX реализованы компоненты, представляющие диаграммы различных типов. Базовым классом для всех видов диаграмм является абстрактный класс javax.scene.chart.Chart. Он определяет три части диаграммы: заголовок, легенду и содержимое диаграммы. Содержимое определяется видом диаграммы (классами-наследниками Chart): секторная диаграмма (класс PieChart) или диаграмма с двумя осями (обобщенный класс XYChart<X,Y>). У класса XYChart<X,Y> есть семь классов наследников: AreaChart, BarChart,

BubbleChart, LineChart, ScatterChart, StackedAreaChart, StackedBarChart (см. документацию [5]).

LineChart<X,Y> представляет линейный график. X задает тип данных по одной оси (оси X), Y - тип данных по второй оси (Y). Собственно оси координат представлены классами, производными от абстрактного класса javafx.scene.chart.Axis<X>: CategoryAxis и ValueAxis<T>. ValueAxis<T> - обобщенный абстрактный класс у которого есть потомок, числовая ось - NumberAxis (class NumberAxis extends ValueAxis<Number>). На графике могут располагаться несколько кривых. Данные для одной кривой представляют серию - объект вложенного класса XYChart.Series<X,Y>. Отдельная точка в серии - это объект вложенного класса XYChart.Data<X,Y>. Например, линейный график с двумя кривыми y=sin(x) и y=cos(x)

```
// Оси координат
NumberAxis xAxis = new NumberAxis();
xAxis.setLabel("Argument");
NumberAxis yAxis = new NumberAxis();
yAxis.setLabel("Function");
// линейный график
LineChart<Number, Number> chart = new LineChart<>(xAxis, yAxis);
chart.setTitle("Chart");
chart.setCreateSymbols(false); /* запретить отображать символы в позициях точек, по которым строится график */
// Серии для графика
XYChart.Series<Number, Number> serie1 = new XYChart.Series<>();
serie1.setName("y(x)=sin(x)"); // будет отображаться в легенде на графике
double x = 0., y = 0.;
…
```

```
y = Math.sin(x);
serie1.getData().add(new XYChart.Data<>(x, y));
…
chart.getData().add(serie1);

XYChart.Series<Number, Number> serie2 = new XYChart.Series<>();
serie2.setName("y(x)=cos(x)");
…
y = Math.cos(x);
serie2.getData().add(new XYChart.Data<>(x, y));
…
chart.getData().add(serie2);
```

Для диаграмм (Chart) не предусмотрены методы задания цвета или толщины кривой, отображаемой на графике. Их можно менять, используя предопределенные классы CSS. Например, в обработчике события можно задать толщину 5 для нулевой (первой по счету) кривой на графике таким образом

```
int lineNumber = 0;
int lineWidth = 5;
chart.lookup(".default-color"+lineNumber+".chart-series-line").
        setStyle("-fx-stroke-width: "+lineWidth+"px;");
```

3.1. Свойства JavaFX

Свойства JavaFX расширяют и улучшают архитектуру компонентов, заданную в технологии JavaBeans. Они являются "наблюдаемыми" (реализуют интерфейс Observable), то есть с ними можно связать блоки прослушивания событий, в первую очередь, связанные с событием изменения значения свойства. Кроме того, свойства JavaFX можно связывать друг с другом таким

образом, что изменение значения одного свойства приводит к изменению другого свойства без явного использования блоков прослушивания событий. В базовом интерфейсе (пакет javax.beans.property) Property<T> объявлены методы:

void bind(ObserableValue<? extends T> o);

позволяет связать свойство, для которого он вызван с другим наблюдаемым объектом (обычно свойством) так, что при изменении второго свойства будет меняться и первое, при этом напрямую изменять первое свойство нельзя;

void bindBidirectional(Property<T> prop);

позволяет связать свойство, для которого он вызван с другим свойством так, что при изменении второго свойства будет меняться и первое, при этом можно напрямую изменять как первое, так и второе свойства;

void unbind();

разрывает связь свойства со связанным на данный момент наблюдаемым объектом;

void bindBiderectional(Property<T> prop);

разрывает связь свойства с указанным свойством;

boolean isBound();

true - если свойство связано с какими-то другими свойствами.

Существуют абстрактные классы для свойств связанных со значениями стандартных типов: BooleanProperty, IntegerProperty, DoubleProperty, StringProperty и т.д. ObjectProperty представляет свойство, связанное с объектом произвольного класса. Для свойства определены методы get и set. Первое возвращает значение, связанное со свойством, например, для свойства IntegerProperty метод get возвращает величину типа Integer. set задает для свойства величину соответствующего типа (например, Integer). Для классов, представляющих свойства конкретных типов есть стандартные реализации -

классы SimpleBooleanProperty и т.д. Существуют также классы, представляющие коллекции наблюдаемых объектов: SimpleListProperty<T> (реализует интерфейс ObservableList<T>), SimpleMapProperty<K,V> (ObservableMap<K,V>).

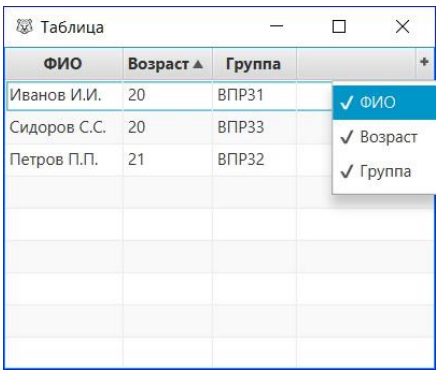
Пример связывания свойств. Есть слайдер, позволяющий выбирать значения из диапазона от 0 до 100 и метка, которая содержит выбранное значение. У слайдера есть свойство value типа DoubleProperty, у метки свойство text типа StringProperty. В JavaFX доступ к свойству должен предоставлять метод, имя которого состоит из имени свойства и слова Property.

```
VBox box = new VBox(5);
Slider slider = new Slider(0, 100, 1); /* минимальное, максимальное и текущее значение */
Label value = new Label("1");
value.textProperty().bind(slider.valueProperty().asString());
box.getChildren().addAll(slider, value);
```

Компонент TableView<T> (аналог Table из Swing). Тип параметра T определяет класс, объекты которого содержат информацию, отображаемую в таблице.

```
class Person {
    private StringProperty name;
    private IntegerProperty age;
    private StringProperty group;
    public Person(String name, int age, String group) {
        this.name.setValue(name);
        this.age.setValue(age);
        this.group.setValue(group);
```

```
        }
        public StringProperty nameProperty() { return name; }
        public IntegerProperty ageProperty() { return age; }
        public StringProperty groupProperty() { return group; }
    }
    …
    TableView<Person> table = new TableView<>();
    ObservableList<Person> data = FXCollections.obseravableArrayList {
```




```
        new Person("Иванов И.И.", 20, "ВПР31"),
        new Person("Петров П.П.", 21, "ВПР32"),
        new Person("Сидоров С.С.", 20, "ВПР33")
        };
        table.setItems(data);
        table.setTableMenuButtonVisible(true);
    TableColumn<Person, String> col1 = new TableColumn<>("ФИО ");
    col1.setCellValueFactory( x -> x.getValue().nameProperty() );
    TableColumn<Person, Integer> col2 = new TableColumn<>("Возраст ");
    col2.setCellValueFactory( x->x.getValue().ageProperty().asObject() );
    TableColumn<Person, String> col3 = new TableColumn<>("Группа ");
    col3.setCellValueFactory(x->x.getValue().groupProperty());
    table.getColumns().addAll(col1, col2, col3);
    …
```

Объект класса ObservableList<T> задает те данные, которые хранятся в модели таблицы (данные таблицы). Как видно из названия класса - это список объектов некоторого класса, связанного с пользовательским типом T, например, Person. Такими классами могут быть, например, классы свойств. Добавление новой строки в таблицу сводится к добавлению нового объекта в список

ObservableList<T> (data в этом примере), удаление - к удалению элемента списка. Можно сделать это и косвенно

```
table.getItems().add(new Person("Mr.X", 20, "XGroup");
int row = 0;
table.getItems().remove(row);
```

Метод setCellValueFactory задает код, который должен сгенерировать значение, отображаемое в ячейках столбца таблицы, для которого вызван этот метод. Аргумент, x в примере выше, содержит ссылку на объект (Person в данном примере), расположенный в редактируемой ячейке.

Если столбцы или строки таблицы визуально не помещаются в окне контейнера, у нее появляются полосы прокрутки.

Для того, чтобы можно было редактировать содержимое ячеек таблицы нужно вызвать метод void setEditable(boolean b) со значением аргумента true:

```
table.setEditable(true);
```

и предоставить компонент, позволяющий пользователю изменять значение в ячейке. Кроме того, после редактирования нужно обеспечить сохранение измененных значений в модели таблицы. Если редактировать содержимое ячейки нужно с помощью стандартного компонента TextField (аналог JTextField в Swing), можно воспользоваться классом javax.scene.control.cell.TextFieldTableCell<S,T>. Его использование приводит к отображению в ячейке поля ввода (объекта класса TextField) в котором и выполняется редактирование.

```
col1.setCellFactory(TextFieldTableCell.forTableColumn());
```

С редактированием ячейки связаны события: start, cancel (потерян фокус ввода), commit (нажата клавиша Enter). Чтобы выполнить свой код-обработчик соответствующего события редактирования ячейки нужно передать его методу void setOnEditX(…), где X это Start, Cancel или Commit.

```
col1.setOnCommit(
        e->{ // e - ссылка на объект, представляющий событие
                int row = e.getTablePosition().getRow();
                Person person = e.getTableView().getItems().get(row);
                person.nameProperty().setValue(e.getNewvalue());
        }
);
```

3.2. Обработка событий

С каждым компонентом связаны какие-то события. Для подключения соответствующих обработчиков события в компонентах предусмотрены методы setOnX(…). Например, ввод текста в тестовое поле и нажатие на клавишу Enter, выбор элемента меню или элемента CheckBox, нажатие на кнопку вызывают возникновение события Action, поэтому у соответствующих компонентов есть метод void setOnAction(EventHandler<ActionEvent> h). В интерфейсе EventHandler<T extends Event> объявлен метод void handle(T event); Таким образом для обработки события нужно предоставить метод с одним аргументом, тип которого совпадает с типом события. Например, кнопка, меняющая свою метку при нажатии на нее:

```
Button btn = new Button("Yes");
btn.setOnAction(
e->{
        if(btn.getText().equals("Yes") btn.setText("No");
        else btn.setText("Yes");
        }
);
```

Кроме того, можно задавать блоки прослушивания для свойств, обрабатывая событие изменения значения соответствующего свойства. Например, у класса Stage есть свойства width и height. Можно добавить блоки прослушивания к этим свойствам чтобы реагировать на изменение ширины и высоты главного окна программы.

```
stage.widthProperty().addListener(
        (o, oldValue, newValue) -> {
                System.out.println("Текущая ширина окна: "+newValue);
// или System.out.println("Текущая ширина окна: "+stage.getWidth());
        }
);
```

3.3. SceneBuilder

Скачать с сайта https://gluonhq.com/products/ scene-builder/ SceneBuilder.

4. 2D графика

Базовый класс javafx.scene.shape.Shape. Производные: Arc, Circle, CubicCurve, Ellipse, Line, Path, Polygon, Polyline, QuadCurve, Rectangle, SVGPath, Text. Класс Shape наследник Node, поэтому все двумерные примитивы можно размещать на сцене и собственно рисование будет сведено к изменениям, применяемым к соответствующим компонентам. В классе Shape определена поддержка атрибутов рисования компонентов: линии контура, заливки внутренней области (если она есть).

```
void setStroke(Paint p);
```

задает цвет контура фигуры, например,

```
double red = .2, green = .3, blue = .4; // значения от 0.0 до 1.0
setStroke(new Color(red, green, blue));
```

в классе Color определено несколько десятков констант, представляющих различные цвета, от обычных, типа Color.RED, Color.GREEN, Color.BLUE до более экзотических, например, Color.ANTIQUEWHITE, Color.CORNSILK, Color.GAINSBORO.

setStroke(Color.GRAY);

статические методы rgb(…) и hsb(…) позволяют создать композитный цвет через красную, зеленую и синюю составляющие цвета или через значения тона (hue), насыщенности (saturation) и яркость (brightness) соответственно

int ired = 20, igreen = 30, iblue = 40, ialpha = 50; // значения от 0 до 255

setStroke(Color.rgb(ired, ogreen, iblue));

setStroke(Color.rgb(ired, ogreen, iblue, ialpha)); // ialpha - прозрачность

можно задавать цвет и так как это принято в web-верстке (на html-страницах), см. документацию [5].

void setStrokeWidth(double width);

задает толщину контура примитива.

void setStrokeType(StrokeType type);

значения перечислимого типа StrokeType: CENTERED (по умолчанию) - линия контура рисуется так, что ее половина находится вне и половина внутри примитива, INSIDE - рамка располагается полностью внутри области занятой примитивом, OUTSIDE - рамка располагается вне области занятой примитивом.

Контур примитива может быть прорисован не сплошной, а пунктирной (штриховой) линией. Задать ее можно так

getStrokeDashArray().setAll(10, 5);

первое значение задает ширину штриха, второе - зазора между штрихами. Можно передавать методу setAll произвольное количество целых значений, например,

getStrokeDashArray().setAll(10, 3, 5, 3);

штрих-пунктирная линия, первый штрих шириной 10 пикселей, второй-шириной 5 пикселей, зазоры между ними по три пикселя, и эта последовательность штрихов и зазоров повторяется вдоль всего контура. Если число фактических аргументов нечетное, при повторе длины штрихов будут задавать размеры зазоров и наоборот, например,

getStrokeDashArray().setAll(10, 3, 5);

вначале идет штрих в 10 пикселей, зазор из 3 пикселей, штрих из 5 пикселей, затем зазор из 10 пикселей, штрих из 3 пикселей, зазор из 5 пикселей и далее все повторяется.

Штрихи могут дополняться (декорироваться) полукругами или прямоугольниками с помощью метода

void setStrokeLineCap(StrokeLineCap type);

значения перечислимого типа StrokeLineCap: BUTT - штрихи не декорируются, ROUND - с левой и правой сторон каждого штриха дорисовывается половина круга, диаметр которого совпадает с толщиной контура, SQUARE - с левой и правой сторон каждого штриха дорисовывается прямоугольник с шириной, равной половине толщины контура и высотой, совпадающей с его толщиной.

void setFill(Paint p);

задает цвет внутренней области фигуры (если такая область есть).

Пример, круг радиусом 30 пикселей, размещается на панели так, чтобы его центр оказался в координатах (100, 120) относительно левого верхнего угла панели, толщина контура - 3 пикселя, цвет контура зеленый, тип контура штриховой, штрихи размером в 3 пикселя с зазорами размером в 3 пикселя, цвет заливки светло-зеленый

Panel panel = new Panel();

Circle circle = new Circle(30); /* радиус 30, можно также использовать метод void setRadius(double r); */

```
circle.setLayoutX(100);
circle.setLayoutY(120);
circle.setStrokeWidth(3);
circle.setStroke(Color.GREEN);
circle.setStrokeLineCap(StrokeLineCap.BUTT);
circle.getStrokeDashArray().setAll(3, 3);
circle.setFill(Color.LIGHTGREEN);
```

Класс javafx.scene.image.Image представляет графические изображения, которые можно загрузить из файлов в формате png, jpg, gif или bmp.

```
Image im = new Image("imp.png");
```

или

```
Image im =
        new Image(getClass().getResource ("img.png").toExternalForm());
```

Класс Image не является наследником класса Node, поэтому для отображения на сцене нужно использовать объекты другого класса javafx.scene.image.ImageView, наследника класса Node.

```
ImageView imgView = new ImageView(im);
```

или

```
ImageView imgView = new ImageView();
imgView.setImage(im);
```

Объект класса ImageView можно разместить на сцене, можно применить к нему преобразования, добавить эффекты. Например, задать горизонтальный и вертикальный размеры

```
imgView.setFitWidth(52);
```

```
imgView.setFitHeight(52);
imgView.preserveRation(true); // сохранить пропорции изображения
// повернуть на заданный угол
imgView.setRotate(90);
```

Можно сделать снимок сцены или ее части (метод snapshot класса Node) и, например, сохранить его в графическом файле формата png, jpg, gif или bmp, для этого используются классы javax.scene.image.WritableImage, javax.imageio.ImageIO, javafx.embed.swing.SwingFXUtils (нужно подключить к проекту модуль javax.swing).

```
…
Pane graphPane = new Pane();
…
File f = new File("img.png");
try {
        if(f != null) {
                WritableImage wi = new WritableImage(imageWidth,
imageHeight);
                graphPane.snapshot(null, wi);
                ImageIO.write(SwingFXUtils.fromFXImage(wi,null),"png",f);
        }
}
catch(IOException ioe) {
        …
}
```

Для рисования в стиле, подобном Swing, если недостаточно возможностей, предоставляемых классами-наследниками Shape, можно использовать

компонент Canvas. С этим компонентом связан графический контекст (GraphicalContext) в котором и производится рисование. Для рисования используются методы GraphicalContext: void strokeText(String text, double x, double y), void strokeLine(...), void strokeOval(...), void strokeRect(...) и т.д., подобные методам void drawString(String text, int x, int y), drawLine, drawOval, drawRect и т.д. класса Graphics из AWT/Swing и методы fillOvlal, fillRect подобные fillOval и fillRect из AWT/Swing. Толщину и цвет линии контура можно задать с помощью методов void setLineWidth(double width) и void setStroke(Paint color), цвет заливки с помощью метода void setFill(Paint color) и т.д. (см. документацию [5]).

```
Canvas canvas = new Canvas();
GraphicsContext gc = canvas.getGraphicsContext2D();
gc.setFill(Color.RED);
gc.fillOval(
        100, 100, /* координаты верхнего левого угла описанного
прямоугольника */
        100, 100, /* координаты верхнего левого угла описанного
прямоугольника */
);
```

5. 3D графика

JavaFX содержит набор предопределенных трехмерных примитивов: Box, Cilinder, MeshView, Sphere, все они наследники стандартного класса javafx.scene.shape.Shape3D, который, в свою очередь, является наследником Node. Можно создавать свои трехмерные примитивы (с помощью MeshView), которые также будет наследниками Shape3D и, тем самым, класса Node. Поэтому они размещаются на сцене так же, как и остальные узлы,

представляющие 2D-примитивы или элементы GUI. В JavaFX 3D используется правая прямоугольная декартовая система координат. Центр ее расположен в верхнем левом углу экрана. Ось OX направлена горизонтально, вдоль верхней рамки экрана, значения на ней возрастают от левого к правому углу экрана. Ось OY направлена вертикально, вдоль левой границы экрана, значения на ней возрастают сверху-вниз. Ось OZ направлена перпендикулярно экрану, за него, то есть вдаль от наблюдателя. Унаследованные от Node методы void setTranslateX(double x), void setTranslateY(double y), void setTranslateZ(double z) и setRotate(int angle) позволяют перемещать 3D-примитивы по сцене и поворачивать их на заданный угол относительно некоторой оси, которая должна быть задана до поворота - void setRotationAxis(Point3D point). Ось направлена из начала координат в направлении, заданном точкой - аргументом setRotationAxis. Point3D - класс, представляющий точку (координаты) в трехмерном пространстве. Shape3D определяет свойство material, определяющее вид 3D-примитива в зависимости от освещенности. Тип свойства - класс, производный от абстрактного класса Material. Существует один такой конкретный класс PhongMaterial.

javafx.scene.shape.Sphere представляет сферу. Конструктор по умолчанию задает сферу единичного радиуса серого цвета с центром, расположенным в начале координат. Конструктор с одним параметром позволяет задать радиус сферы. Радиус сферы - это свойство (DoubleProperty).

javafx.scene.shape.Cylinder представляет цилиндр. Конструктор по умолчанию задает цилиндр единичного радиуса высотой 2 серого цвета с центром, расположенным в начале координат. Конструктор с двумя параметрами позволяет задать радиус и высоту цилиндра. Радиус и высота цилиндра - это свойства (DoubleProperty).

javafx.scene.shape.Box представляет параллелепипед. По умолчанию создается куб размерами 2x2x2. Свойства: width, height, depth (DoubleProperty).

На сцене с 3D-объектами по умолчанию присутствует ортогональная камера и источник света. Ортогональная камера выполняет отображение трехмерной сцены на поверхность экрана без учета перспективы, то есть степени удаленности объектов от экрана (плоскости XY). Можно задать камеру, учитывающую перспективу (класс javax.scene.PerspectiveCamera) для обеспечения картинке объема.

Пример, сцена с двумя сферами и цилиндром

```
…
public class Lab1_6 extends Application {
        …
        private javax.scene.Group molecule = new Group();
        private double r = 50;
…
        Sphere m1 = new Sphere(r), m2 = new Sphere(r);
        Color curColor = Color.rgba(0xb3, 0x66, 0x1a, 1);
        m1.setMaterial(new PhongMaterial(curColor));
        m2.setMaterial(new PhongMaterial(curColor));
        m1.setTranslateX(100);
        m2.setTranslateX(300);
        m1.setTranslateY(100);
        m2.setTranslateY(100);
        m1.setTranslateZ(0);
        m2.setTranslateZ(0);
 /* можно задать режим рисования: сплошной (по умолчанию), каркас из линий
(LINE) */
```

```
 // m2.drawModeProperty().set(DrawMode.LINE);
double h = 200-2*r+4; // высота цилиндра, соединяющего сферы
Cylinder bond12 = new Cylinder(r/3,h);
bond12.setMaterial(new PhongMaterial(curColor));
bond12.setTranslateX(100+r+h/2-2);
bond12.setTranslateY(100);
bond12.setTranslateZ(0);
bond12.setRotate(90); // изначально цилиндр расположен вертикально

molekule.getChildren().addAll(m1, m2, bond12);

double xAngle = 20, yAngle = 33, zAngle = 45;
javafx.scene.transform.Rotate xRotate = new Rotate(0, Rotate.X_AXIS);
Rotate yRotate = new Rotate(0,Rotate.Y_AXIS);
Rotate zRotate = new Rotate(0,Rotate.Z_AXIS);

xRotate.setAngle(xAngle);
yRotate.setAngle(yAngle);
zRotate.setAngle(zAngle);

box.getTransforms().clear();
box.getTransforms().addAll(xRotate, yRotate, zRotate);
…
```

6. Мультимедиа

Компонент javax.scene.media.Media представляет в программе мультимедийный ресурс. При создании объекта Media указывается путь к

ресурсу (с помощью универсального идентификатора ресурса URI, в локальной файловой системе просто путь к файлу), с которым он будет связан. Через объект Media можно получить информацию о продолжительности воспроизведения медиа потока, дорожках и видео-разрешении.

```
String filePath = "file.mp4";
Media media = null;
try {
    URI uri = Paths.get(filePath).toAbsolutePath().toUri();
    media = new Media(uri.toString());
} catch(Exception ex) {
    …
}
```

Компонент MediaPlayer представляет медиа проигрыватель, которому передается мультимедийный ресурс (Media) для отображения. Метод Media getMedia() класса MediaPlayer позволяет получить ссылку на медиа ресурс заданный плееру. Через эту ссылку (когда плейер готов к воспроизведению) можно получить, например, размеры видео (методы getWidth() и getHeight() класса Media). Методы класса MadiaPlayer - void play(), void pause(), void stop(), void seek(Duration d) позволяют соответственно запустить, приостановить, остановить воспроизведение мультимедиа потока, перейти к заданной позиции в потоке. Duration представляет некоторый интервал времени. В этом классе предусмотрен один конструктор Duration(double value), value задает число миллисекунд в интервале. Есть методы Duration add(Duration d), Duration subtract(Duration d), Duration multiply(double d), Duration divide(double d), double toHours(), double toMinutes(), double toSeconds(), double toMillis(), int compareTo(Duration d).

```
MediaPlayer mplayer = new MediaPlayer(media);
```

Компонент MediaPlayer не является наследником Node, поэтому для размещения его на сцене нужно передать его объекту класса MediaView.

```
MediaView mview = new MediaView(null);
mview.setMediaPlayer(mplayer);
/*
или
MediaView mview = new MediaView(mplayer);
*/
Pane pane = new Pane();
pane.getChildren().add(mview);
```

Получить ссылку на объект класса MediaPlayer через объект класса MadiaView можно с помощью метода getMdiaPlayer().

7. Анимация

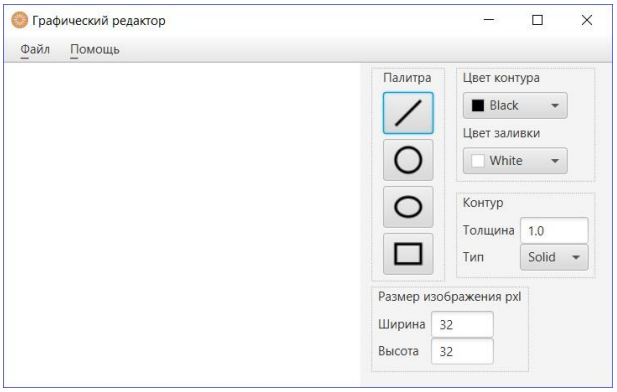
см. классы Timeline, KeyFrame, KeyValue [5].

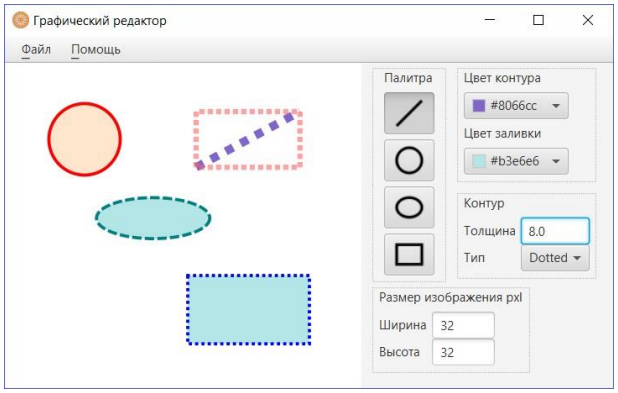
ЗАДАНИЕ 1. Создайте JavaFX-приложение, содержащее в главном окне панель, кнопку "Сохранить" и группу кнопок с зависимой фиксацией (ToggleButton), расположенных вертикально. Каждая кнопка должна содержать какое-то изображение. Найдите или создайте для этого графические (jpg, png, gif, …) файлы небольшого размера, например, 64x64 пикселя. Если какая-то из кнопок ToggleButton выбрана, щелчок правой кнопкой мыши внутри панели должен приводить к отображению в позиции курсора такой же картинки, что и на кнопке. Все изображения, полученные после щелчков мышью,

накапливаются, образуя некоторую составную картинку. Нажатие на кнопку "Сохранить" должно приводить к выводу стандартного диалога SaveDialog и к дальнейшему сохранению составной картинки в указанном пользователем файле.

ЗАДАНИЕ 2. Создайте JavaFX-приложение в окне которого отображается один из следующих графических 2D-примитивов, выбранный случайным образом: отрезок прямой, окружность, эллипс или прямоугольник. Вокруг примитива должна отображаться полупрозрачная рамка с пунктирными границами (описанный прямоугольник). Пользователь должен иметь возможность управлять расположением на экране и размерами отображенного примитива с помощью клавиатуры. Клавиши со стрелками должны позволять перемещать примитив (и его рамку) по окну приложения, клавиши + и – менять вертикальный (по сути сдвигать нижнюю границу прямоугольника, в который вписан примитив), а клавиши < и > менять горизонтальный размер (сдвигать правую границу рамки).

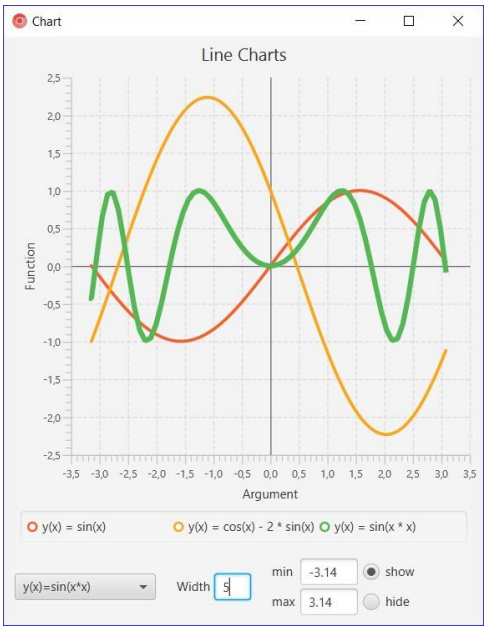
ЗАДАНИЕ 3. Создайте JavaFX-приложение "Графический редактор". Главное окно приложения должно выглядеть примерно так:





Поддерживаемый функционал: в палитре можно выбрать примитив (группа кнопок ToggleButton), затем, по щелчку правой кнопкой мыши в свободной области окна (панели отображения рисунка) появляется изображение выбранного примитива с цветом, толщиной и типом (сплошная, пунктирная, точечная) границы, указанными в соответствующих элементах управления правой панели (панели управления). Для всех примитивов, кроме отрезка прямой используется указанный на панели управления цвет заливки внутренней области. Вокруг примитива отображается полупрозрачная рамка, показывающая, что можно изменять его положение в панели отображения рисунка и изменять его размеры. Изменение размеров выполняется как в задании 2. Все изменения можно проводить только с текущим примитивом, но сохраняются и все добавленные раньше. Главное меню программы позволяет сохранить картинку в выбранном пользователем файле (Файл-Сохранить…), выйти из программы (Файл-Выход) и получить справку о том, как использовать программу (Помощь-Инструкция). При сохранении в файл используются размеры картинки (в пикселях), указанные в панели управления.

ЗАДАНИЕ 4. Создайте JavaFX-приложение для отображения графиков функций. Используйте SceneBuilder для создания внешнего вида приложения и добавления нужных компонентов, включая LineChart. Приложение должно позволять выбирать для отображения функции из некоторого фиксированного набора. Для графика функции должны задаваться толщина линии, диапазон значений аргумента, шаг (постоянный) изменения аргумента. Должна быть возможность одновременного отображения графиков нескольких функций. Для каждой из таких функций нужно предусмотреть возможность выбора - должен график отображаться вместе с другими или нет.



ЗАДАНИЕ 5. Создайте JavaFX-приложение для отображения данных в виде таблицы. Используйте компонент TableView. Задайте для таблицы заголовки, например, "Язык", "Автор" и "Год". Поместите в нее данные о языках программирования – название, автор, год появления, например, Си, Деннис Ритчи, 1972; C++, Бьерн Страуструп, 1983; Python, Гвидо ван Россум, 1991; Java, Джеймс Гослинг, 1995; JavaScript, Брендон Айк, 1995; C#, Андерс Хейлсберг, 2001; Scala, Мартин Одерски, 2003. Таблица должна позволять добавлять новые данные (строки), изменять имеющиеся, выбирать, какие столбцы показывать, а какие нет. Щелчок мышкой по заголовку столбца таблицы должен приводить к тому, что данные в ней выводятся в отсортированном по значениям этого столбца виде (по возрастанию или по убыванию). При этом после заголовка этого столбца должен появится значок в

виде стрелки или треугольника, показывающий в каком направлении выполнена сортировка (по возрастанию или по убыванию). Повторный щелчок по тому же самому заголовку должен менять порядок сортировки.

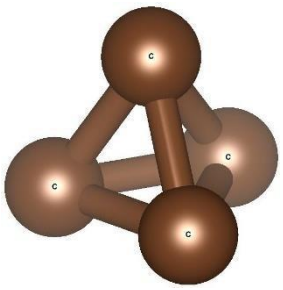
ЗАДАНИЕ 6. Создайте JavaFX-приложение для 3D-визуализации молекул. В текстовом файле с расширением xyz содержится информация о молекуле в следующем формате:

4

C4

C	0.934285	0.231605	-0.978735
C	1.14877	0.131834	0.119626
C	1.98794	0.122377	-0.623269
C	1.43534	1.08196	-0.425853

Здесь 4 - количество атомов в молекуле, C4 - произвольная строка с описанием



молекулы, далее четыре строчки с описанием четырех атомов из которых состоит молекула. Описание атома состоит из его названия (таблица Менделеева, например, C-углерод) и трех координат в прямоугольной декартовой системе координат. Пользователь выбирает нужный ему xyz-файл и в окне программы отображается молекула. Причем атомы изображаются как закрашенные сферы. Ближайшие атомы соединяются линией или цилиндром. Цвет и размер сфер разные для атомов разного типа, но одинаковые для атомов одного и того же типа. Программа должна позволять пользователю изменять отображаемый в окне размер молекулы, вращать ее относительно осей координат, задавать цвет для разных атомов. Кроме того, должна быть возможность сохранять картинку в графическом файле (png, jpg, gif).

Как построить цилиндр по двум 3D-точкам см.
https://fooobar.com/questions/12325117/javafx-3d-transforming-cylinder-to-defined-start-andend-points

public Cylinder createConnection(Point3D origin, Point3D target) {

Point3D yAxis = new Point3D(0, 1, 0); / цилиндр изначально расположен вертикально (высота вдоль оси OY), направляющий вектор для оси OY - (0, 1, 0) */*

Point3D diff = target.subtract(origin); / разность векторов target и origin - вектор, направленный от origin к target */*

double height = diff.magnitude(); / расстояние между origin и target - высота цилиндра */*

Point3D mid = target.midpoint(origin); / точка, лежащая посередине между target и origin - сюда нужно переместить цилиндр (поместить его центр) */*

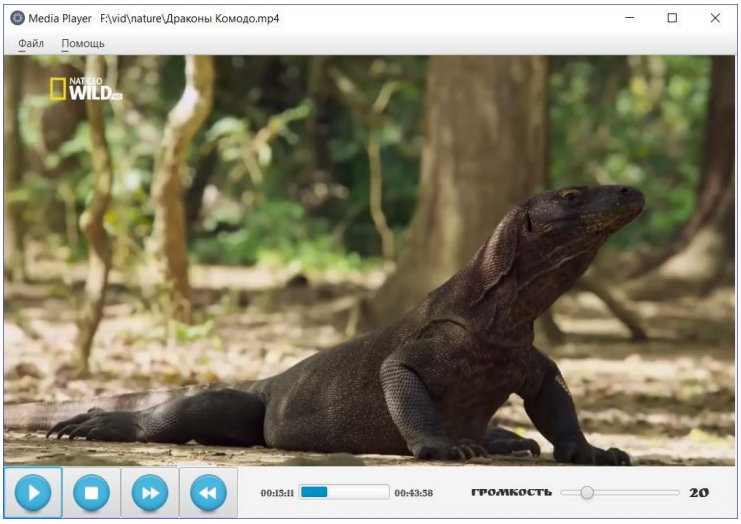
Translate moveToMidpoint = new Translate(mid.getX(), mid.getY(), mid.getZ());

Point3D axisOfRotation = diff.crossProduct(yAxis); / ось, вокруг которой нужно повернуть цилиндр - направлена перпендикулярно плоскости, в которой лежат пересекающиеся вектора diff (направление от origin к target) и yAxis (текущее направление высоты цилиндра), получается как векторное произведение diff и yAxis */*

double angle = Math.acos(diff.normalize().dotProduct(yAxis)); / угол поворота цилиндра - угол между нормализованным (длина равна 1) вектором diff и вектором yAxis */*

```
        Rotate rotateAroundCenter = new Rotate(-Math.toDegrees(angle),
axisOfRotation);
        Cylinder line = new Cylinder(1, height); /* радиус цилиндра 1, нужно
заменить на свое значение */
        line.getTransforms().addAll(moveToMidpoint, rotateAroundCenter);
        return line;
}
```

ЗАДАНИЕ 7. Создайте JavaFX-приложение "Media Player". Приложение должно позволять пользователю открыть мультимедиа файл, например, в формате mp4, показать пользователю его содержимое. У пользователя должна быть возможность запустить, приостановить, продолжить воспроизведение, изменить громкость звучания. Должны отображаться: название отображаемого файла, общая длительность воспроизводимой записи, длительность уже воспроизведенного участка.



(не обязательное) ЗАДАНИЕ 8. Создайте JavaFX-приложение "Test Animation" показывающее различные эффекты анимации. Используйте возможности классов Animation, TimeLine, различных классов Transformation и т.п. Набор демонстрируемых эффектов и элементы GUI для их запуска/настройки придумайте самостоятельно.

Литература

1. "JavaFX getting started with JavaFX, Release 8". - Oracle, 2008, 20014. - 68 p. https://docs.oracle.com/javase/8/javafx/get-started-tutorial/index.html
2. S.Grinev. "Mastering JavaFX 10". – Packt Publishing. – 2018, 245c.
3. C.Dea, G.Grunwald, J.Pereda, S.Phillips, M.Heckler "JavaFX 9 by example. 3ed.". – Apress. – 2017, 557 p.
4. Н.А.Прохоренок "JavaFX". – СПб.:BHV-Петербург. – 2020, 768 с.
5. JaxaFX Documentation. https://gluonhq.com/products/javafx/

Редактор А.А. Литвинова

ЛР № 04779 от 18.05.01.	В набор	В печать
Объем 0,5 усл.п.л., уч.-изд.л.	Офсет.	Формат 60x84/16.
Бумага тип №3.	Заказ №	Тираж 120. Цена

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:

344010, г. Ростов-на-Дону, пл. Гагарина, 1.